

Autoscribe Architecture Recap — March 7

Conversation

This document summarizes the architectural decisions reached regarding Studio registries, Autoscribe configuration storage, and database structure. It is intended as an implementation reference when returning to Autoscribe development.

Core Principle

The system separates three concerns:

- 1 . **Configuration** — what the pipeline should do
- 2 . **Execution history** — what the pipeline actually did
- 3 . **Runtime coordination** — what the pipeline is doing right now

This leads to a clean three-layer architecture.

System Layers

Studio

Human-editable configuration and editorial environment.

Contains:

- templates
- registry YAML files
- prompt files

Studio is the **editorial source of truth**.

Example structure:

Studio/ _common/ templates/ registry/

Registry files may include:

- editorial classes
- template mappings

- folder mappings
 - pipeline verbs
-

Autoscribe Databases

Two SQLite databases are used.

1 . pipeline.db

Purpose: **pipeline configuration**

Tables:

instructions verbs

These define how Autoscribe behaves.

Example instructions schema:

instruction_slug vault_slug body created updated

Primary key:

(instruction_slug, vault_slug)

Example verbs schema:

verb_slug vault_slug description engine parameters (JSON) parent_verb created updated

Primary key:

(verb_slug, vault_slug)

Verbs are scoped by vault exactly like instructions.

Lookup order:

- 1 . verb / vault
- 2 . verb / gbl

Inheritance can optionally resolve nested verbs.

Example:

rewrite.voice

Resolution:

rewrite.voice / vault rewrite.voice / gbl rewrite / vault rewrite / gbl

2 . ledger.db

Purpose: **execution history**

Tables:

call_records responses

The ledger records everything that actually happened during execution.

Example call_records schema:

call_ulid (PRIMARY KEY) batch_slug step_slug input_content origin_blob created

Example responses schema:

call_ulid (PRIMARY KEY) batch_slug response_content results_blob created

Join relationship:

responses.call_ulid → call_records.call_ulid

This creates a strict **one-to-one relationship** between calls and responses.

Batch Slug Strategy

`batch_slug` appears in both tables.

Benefits:

- entire pipeline runs can be reconstructed from the ledger
- foreign key joins remain simple
- batch queries remain efficient

Recommended indexes:

```
CREATE INDEX idx_calls_batch ON call_records(batch_slug);
```

```
CREATE INDEX idx_responses_batch ON responses(batch_slug);
```

Redis Role

Redis handles **runtime coordination only**.

Examples:

- worker queues
- job state
- batch lifecycle

Batch registry entries are ephemeral.

Example key:

batch:<batch_slug>

Redis TTL can be long (e.g., one month) but the system must not depend on Redis for reconstruction.

If Redis disappears, the ledger still contains all information needed to analyze runs.

Resulting Architecture

Configuration layer:

Studio files pipeline.db

Execution history:

ledger.db

Runtime state:

Redis

Design Philosophy Reinforced

The architecture follows the principle:

"Artifacts, not flags, are the source of truth."

Persistent artifacts:

- instructions
- verbs
- call records
- responses

Ephemeral coordination:

- Redis runtime state
-

Final Directory Layout Example

Autoscribe/

db/ pipeline.db ledger.db

workers/ ingest/ compiler/

Studio/

_common/ templates/ registry/

Key Outcomes From This Discussion

- 1 . Verbs moved from Redis to database storage
 - 2 . Instructions and verbs stored in **pipeline.db**
 - 3 . Execution history stored in **ledger.db**
 - 4 . Batch registry remains ephemeral in Redis
 - 5 . `batch_slug` stored in ledger tables for reconstruction
 - 6 . Redis only manages runtime coordination
 - 7 . Studio remains the editable configuration environment
-

Implementation Order (Suggested)

- 1 . Rename instructions DB → `pipeline.db`
- 2 . Create `verbs` table
- 3 . Ensure vault scoping matches instruction lookup logic
- 4 . Confirm ledger schema includes `batch_slug`
- 5 . Add batch indexes
- 6 . Remove any remaining verb definitions from Redis

End of recap.